

Syntax-Based Calibration for Pretraining Code Detection on Large Language Models

Anonymous submission

Abstract

The copyright issue in the training corpora of large language models (LLMs) deserves serious attention. Respecting copyright and intellectual property is essential, not only as a legal obligation, but also as a means of fostering a supportive and respectful environment for content creators. Particularly, we focus our attention on copyright (or copyleft) protection of code used during the pre-training process of LLMs.

we investigate the ownership of the copyright.

Introduction

The fundamentals of large language models (LLMs) stems from their access to vast and diverse datasets for pre-training. However, the copyright issue in the training corpora deserves serious attention. In recent years, numerous legal disputes have arisen concerning the use of copyrighted materials in training data (Grynbaum and Mac 2024; Vynck 2025). Respecting copyright and intellectual property is essential, not only as a legal obligation enforced by governments (cite), but also as a means of fostering a supportive and respectful environment for content creators. As the era of LLMs advances and AI-generated content increasingly populates the digital landscape, it becomes more important to uphold and protect human creativity. Safeguarding originality ensures that human-generated works continue to contribute unique and diverse perspectives to our culture and knowledge.

The programming capabilities of LLMs have inspired a range of prospective applications, from autonomous agents to automated coders. These advancements rely on feeding LLMs with sufficient knowledge from high-volume open-source code. However, such code is typically governed by specific usage obligations. *e.g.*, copyleft licenses like the GNU General Public License (GPL). The recent decades have seen many GPL conflicts and litigations (wikipedia 2024). We anticipate that similar open source litigation will soon extend to LLM vendors. Looking back, the GPL was introduced in 1989 as part of the GNU Project (?). Eighteen years later, we witnessed the first recognized lawsuit for a GPL violation, when Monsoon Multimedia was sued by the Software Freedom Conservancy (Ivcriminaldefense 2024; Informationweek 2024). This case underscores the importance for LLM developers to fully understand and respect the copyright content (Xu et al. 2024). Unfortunately,

conflicts surrounding the use of pretraining data continue to emerge (cite), and few have reached resolution due to the protracted nature of litigation.

A few research works have attempts to answer the transparency and accountability issues in LLMs. Zhou et al. (Zhou et al. 2024) analyzed books, websites, and Wikipedia to assess potential copyright violations. Recent benchmarks (Chen et al. 2024; Liu et al. 2024) support evaluation of both literal copying (*e.g.*, verbatim reproduction) and non-literal copying (*e.g.*, replicated plots or characters) from LLM outputs. There are also proactive measures known as copyright traps (Shilov, Meeus, and de Montjoye 2024; Meeus et al. 2024), which embeds carefully crafted copyrighted content (*i.e.*, similar to watermark) to enable precise detection of copyright violations. However, this approach inherently suffers from the limitation that such traps must be prepared in advance.

A more promising approach involves applying membership inference attacks (MIAs) to LLMs. MIAs were initially used to determine whether a specific data point was part of the training dataset of a machine learning model (Shokri et al. 2017; Song and Mittal 2021; Song, Shokri, and Mittal 2019; Huang, Gong, and Reiter 2024; Yeom et al. 2018; Leino and Fredrikson 2020; Long et al. 2018; Nasr, Shokri, and Houmansadr 2019; Sablayrolles et al. 2019; Salem et al. 2018; Truex et al. 2019), thereby exposing potential privacy risks. MIAs in LLMs share the same intuitions that models memorizes data (overfitting) (Song and Shmatikov 2019; Carlini et al. 2021). Tirumala et al. (Tirumala et al. 2022) find that models memorize nouns and numbers, and that larger models can memorize a larger portion of the data before overfitting. **Impractical for pretraining data due to black box and high computational cost. LLM unavailability of pretraining data distribution.**

Unlike machine learning MIAs that rely on shadow or reference models, MIAs in LLMs typically exploit the model’s token-level output and probabilities. Carlini et al. (Carlini et al. 2021) demonstrated that LLMs can leak verbatim training sentences. Nevertheless, identifying non-literal memorization remains more challenging compared to literal (verbatim) memorization. **Most approaches focus on modeling the likelihood of word sequences in context.** Duan (Duan et al. 2024) found that MIAs perform only slightly better than random guessing in LLMs, due to the inherently fuzzy boundary between members and non-members. Several approaches

have been proposed, including leveraging neighboring data via synonym substitution (Mattern et al. 2023), detecting low-probability tokens (Shi et al. 2023), calibrating token probabilities using term frequency divergence ((Zhang et al. 2024)). However, they focus on general texts instead of code.

Yang et al. (Yang et al. 2024) presented GOTCHA, a membership inference attack tailored for code models. Similarly, Wan et al. (Wan et al. 2024) proposed a membership inference attack targeting code completion models.

Our method based on assumption that: language grammar require certain must-appear-after patterns, that elicit key word pairs, where the language models remember for this language and tend to have high probabilities if keyword appears.

We argue from the essential of copyright. The keywords in the code are inks and brushes, which should not be considered for MIA computation.

Like prior works (Duan et al., 2024; Shi et al., 2024), we assume that we only have access to the target LLM as a black box: we can compute token probabilities for the text to be detected but have no access to the internals of the LLM (e.g., weights and activations).

To facilitate this study and the evaluation of pre-training data detection for LLMs, we introduce a new benchmark named PatentMIA, specifically designed for Chinese-language pretraining data detection. PatentMIA is sourced from Google- Patents (Google, 2006) and constructed following Shi et al. (2024), who distinguish between training and non-training data based on cut-off dates of the target LLM, where training data precedes, and non-training data follows, the cut-off date.

Challenge I. Benchmark how to determine member and non-member.

Challenge II. code syntax.

challenge: verbatim instances

Contribution. We summarize our contributions as follows:

Related work

MIA in Machine Learning. Membership inference attacks (MIAs) have gained traction as tools to audit privacy risks in machine learning (Song and Mittal 2021; Song, Shokri, and Mittal 2019; Huang, Gong, and Reiter 2024; Yeom et al. 2018; Leino and Fredrikson 2020; Long et al. 2018; Nasr, Shokri, and Houmansadr 2019; Sablayrolles et al. 2019; Salem et al. 2018; Truex et al. 2019). Shokri et al. (Shokri et al. 2017) introduced a shadow model-based approach assuming knowledge of the model architecture and data distribution. Subsequent works relaxed or challenged these assumptions. Salem et al. (Salem et al. 2018) used diverse shadow model architectures; Liu et al. (Liu et al. 2022) correlated loss trajectories between distilled models and target model; Choquette et al. (Choquette-Choo et al. 2021) assumed confidence scores are unavailable. Rezaei et al. (Rezaei and Liu 2021) highlighted evaluation pitfalls between overfitted and well-trained models. Kazmi et al. (Kazmi et al. 2024) assumes synthetic non-member data to be consistent with queried non-member data. Ye et al. (Ye et al. 2022) formalized attack assumptions in a unified framework. Besides, the shadow model framework has seen other

advances. Yeom et al. (Yeom et al. 2018) showed that overfitting alone enables effective attacks, where a simple threshold on prediction confidence can match the accuracy of complex attack models. Song et al. (Song et al. 2021) systematic highlighted the lack of per-sample analysis, prompting difficulty calibration methods that adjust membership scores based on sample hardness (Watson et al. 2021; He et al. 2024). Carlini et al. (Carlini et al. 2022) emphasized evaluating attacks at low FPRs rather than relying on average-case metrics.

MIA in Language Models. Song et al. (Song and Shmatikov 2019) investigated how deep learning-based text generation models memorize training data, while Carlini et al. (Carlini et al. 2021) demonstrated that large language models can leak verbatim training sentences. While literal (verbatim) memorization is easier to detect, identifying non-literal memorization remains challenging. Most approaches focus on modeling the likelihood of word sequences in context. Duan (Duan et al. 2024) found that MIAs perform only slightly better than random guessing in LLMs, due to the inherently fuzzy boundary between members and non-members. To address this, Mattern et al. (Mattern et al. 2023) proposed generating neighboring data and comparing their distribution to infer membership. Zhang et al. (Zhang et al. 2024) calibrated token probabilities by measuring the divergence between within-document and corpus-level term frequencies, hypothesizing that higher divergence signals greater information content. Shi et al. (Shi et al. 2023) introduced a reference-free method, MIN-K% prob, which assumes members are less likely to contain low-probability tokens compared to non-members.

Copyright Sources in Language Models. Xu et al. (Xu et al. 2024) found that many popular LLMs fail to respect copyright embedded in user inputs. Zhou et al. (Zhou et al. 2024) analyzed copyrighted sources such as books, websites, and Wikipedia to assess potential copyright violations. Recent benchmarks (Chen et al. 2024; Liu et al. 2024) support evaluation of both literal copying (e.g., verbatim reproduction) and non-literal copying (e.g., replicated plots or characters) in LLM outputs. Documents memorization in LMs is another challenge. Meeus et al. (Meeus et al. 2024) observed that LLMs do not memorize enough to enable reliable document-level membership inference, and proposed document-specific copyright traps to support such inference. Shilov et al. (Shilov, Meeus, and de Montjoye 2024) introduced fuzzy copyright traps designed to evade deduplication and persist through training. Interestingly, despite the well-established nature of code copyright—supported by numerous litigations and enforcement cases—there has been relatively little research on copyright issues in code-generating LLMs. Yang et al. (Yang et al. 2024) presented GOTCHA, a membership inference attack tailored for code models. Similarly, Wan et al. (Wan et al. 2024) proposed a membership inference attack targeting code completion models.

Methodology

We discuss the motivation of our method design as the preliminary. Then we elaborate our method ECHOSRC.

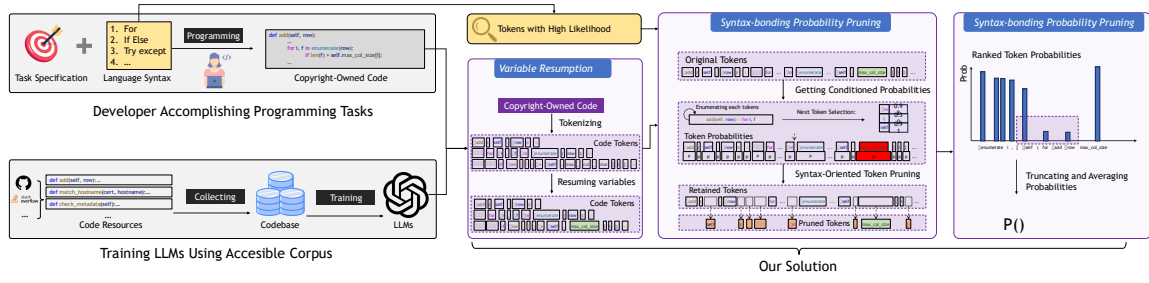


Figure 1: Overview

Preliminary

Source code is considered a “literary work” under copyright law. The “literary work” can be viewed as a sequence of tokens arranged to reflect the author’s intentions. Existing MIA techniques [CITE](#) treat all source code tokens equally and compute membership probabilities of a LLM based on a sample of these tokens. Generally, it is recognized that elements such as variable names, data and control flows, and API usage together with the involved variables, embody the author’s intent and serve as indicators of source code authorship. However, we observe that some tokens primarily serve a grammatical purpose, functioning to complete the syntax constraint of the programming language.

Syntax Constraints

We focus on the Python language. Python is a popular programming language with wide applications, including data analysis, artificial intelligence, and scientific computing. To build our syntax pattern database, we combine manual inspection of the official documentation (Foundation 2025b; Oracle 2025) with automated analysis.

Specifically, we systematically survey the language specification, focusing on Chapter 8, “Compound statements,” which defines the core control flow and organizational structures. This chapter’s 10 subsections detail the multi-line constructs rich in syntactic patterns relevant to our work.

Our selection process is grounded in Python’s lexical structure. We programmatically retrieve reserved keywords using the `keyword` module (35 in Python 3.11) to ensure our patterns are based on definitive language elements. Based on this, we analyzed the 10 subsections and included 8 in our database: `if`, `for`, `try`, `with`, `match`, `function` (`def`) and `class` (`class`) definitions, and `coroutine` (`async`) structures. These were selected because their syntactic forms are rigid multi-token patterns, identifiable via tokenization, that contain both prefix keywords (e.g., `for`, `try`) and structurally determined suffix tokens (e.g., `in`, `except`).

We excluded two subsections. The `while` statement was omitted as its core structure (`while condition:`) lacks a fixed suffix token. “Type parameter lists” was also excluded as it pertains to type-level declarations, not the executable control-flow syntax central to our study. The syntactic validity of all collected patterns is finally verified using Python’s `ast` module.

The inherent structural duality of source code – simultaneously embodying *human design intent* and *machine-enforced*

syntax – introduces unique challenges for membership inference in code language models.

continue Our analysis of XXX Python projects reveals that XXX% of code tokens are bound by syntactic constraints (e.g., `for-in`, `try-except`), forming rigid patterns that obscure the memorization signals crucial for membership detection. This syntactic determinism manifests two fundamental barriers:

- **Syntax-Induced Noise Amplification:** Language-mandated tokens like `in`, `except`, and `:` exhibit near-identical probability distributions across member and non-member samples. Their structural predictability drowns out discriminative signals from author-specific patterns.
- **Contextual Ambiguity:** Traditional token-level approaches cannot distinguish between *syntax-enforced occurrences* (e.g., `in in for x in data`) and *semantically meaningful instances* (e.g., `if key in dict`). This equivalence creates false attribution pathways.

The root challenge stems from code’s *mandatory scaffolding* property: any functionally correct implementation must embed programmer intent within grammatical constraints. Whereas natural language allows infinite paraphrasing, code requires specific token sequences to satisfy the interpreter. This creates an adversarial scenario where the very tokens that define correctness (keywords, delimiters) act as persistent noise sources for membership inference.

Our approach leverages the *dual predictability hierarchy* inherent in code:

Language-Level Predictability: Syntax-enforced tokens follow deterministic patterns dictated by Python’s grammar

Author-Level Predictability: Unique implementation choices (variable names, API chains) reflect training data memorization

By surgically separating these predictability layers through syntax-aware filtering, we amplify the discriminative power of membership signals while suppressing structural noise. This paradigm shift addresses the core limitation of existing methods – their inability to account for code’s machine-enforced regularity – establishing a foundation for code-specific membership inference.

Min-k% Prob with Syntax-Aware Filtering

We propose **Syntax-Aware Min-k% Prob**, an enhanced membership inference method specifically designed for code data. This extension addresses the fundamental difference

between natural language and programming languages: code contains *syntax-enforced patterns* where specific token sequences are structurally determined (e.g., `for...in...`, `if...else...`). The core insight is that suffix tokens in these patterns (e.g., `in`, `else`) exhibit high prediction probabilities regardless of their membership status, introducing noise to the Min-k% Prob metric. Our method introduces a two-stage filtering process:

1. **Syntax Rule Construction:** Establish a comprehensive syntax rule set $\mathcal{R}$ based on Python’s formal grammar specification. For each syntax-bound phrase $r_i \in \mathcal{R}$, we define:

$$r_i = (\phi_{\text{prefix}}, \phi_{\text{suffix}}, \tau)$$

where:

- ϕ_{prefix} : Context-sensitive prefix tokens (e.g., `for`, `try`)
- ϕ_{suffix} : Structurally determined suffix tokens (e.g., `in`, `except`)
- τ : AST node type for validation (e.g., `For`, `Try`)

Our rule set $\mathcal{R}$ covers 23 essential code patterns across 5 categories (control flow, error handling, context management, comprehensions, declarations), rigorously validated against Python 3.11 documentation.

2. **Hybrid Pattern Detection:** For input code x , perform dual validation:

- (a) *Surface-Level Matching:* Scan tokens using regular expressions derived from ϕ_{prefix} and ϕ_{suffix}
- (b) *Semantic Validation:* Construct Abstract Syntax Tree (AST) to verify pattern validity through node type checking with τ

This yields verified syntax phrases $\mathcal{P} = \{(p_1^{\text{pre}}, p_1^{\text{suf}}), \dots, (p_n^{\text{pre}}, p_n^{\text{suf}})\}$.

3. **Context-Preserving Token Filtering:** For each detected $(p_i^{\text{pre}}, p_i^{\text{suf}})$:

$$\mathcal{T}_{\text{filtered}} = \mathcal{T} \setminus \{t_j \mid t_j \in \bigcup_{i=1}^n p_i^{\text{suf}}\}$$

preserving contextual relationships through:

4. **Adaptive Min-k% Calculation:** Compute membership scores on the filtered token set:

$$\text{Syntax-Min-k\%}(x) = \frac{1}{|\mathcal{S}|} \sum_{t_i \in \mathcal{S}} -\log p(t_i | t_{<i})$$

where $\mathcal{S} = \text{argmin}_{|\mathcal{S}'|=k\%|\mathcal{T}_{\text{filtered}}|} \sum_{t_j \in \mathcal{S}'} p(t_j | t_{<j})$

Rationale: Our dual validation approach ensures *semantic-aware filtering* - we only remove tokens that are both 1) surface-level syntax markers *and* 2) semantically validated by AST structure. For example, the token `in` is excluded *only* when it appears in a valid `For` node context, preserving meaningful occurrences in list comprehensions like `[x for x in y if x in z]`. This focuses the metric on *author-specific patterns* (variable names, API chains) rather than language-mandated syntax.

Table 1: Benchmark Statistics

Lang.	Label	Proj. (#)	Files (#)	Func. (#)	LOC (#)	Length
Python	Members	Pile	-	1000	-	100
						200
						300
	Non-Members	100	-	1000	-	100
						200
						300

Table 2: Top 10 AST Tokens in our Benchmark

Language	AST Token	Count
Python	Load	31,628
	Name	25,705
	Constant	12,404
	Call	8,516
	Attribute	8,409
	Store	5,471
	Assign	4,043
	keyword	3,340
	Expr	3,177
	Subscript	1,528
	Total	123,681

Benchmark Construction

Existing MIA benchmarks, e.g., WikiMIA (Shi et al. 2023) and MIMIR (Duan et al. 2024), are designed to evaluate MIAs on generic data elements. They can not accurately capture MIA capabilities on code or the extent to which LLMs utilize copyrighted code. **Although the benchmark from Yang et al. (Yang et al. 2024) targeted code, it has several limitations for our study. Their benchmark only covers the Java language and relies on the general-purpose CodeXGLUE (Lu et al. 2021). Furthermore, their MIA method requires training an auxiliary classifier, which constitutes a complete attack pipeline. Our study, however, requires a purer benchmark consisting only of member and non-member samples to directly evaluate our method’s performance on the LLM, rather than assessing a complex attack process involving auxiliary models. Our time-based construction of the benchmark is therefore more direct and better aligned with the goal of evaluating the model’s "memory".** Therefore, we constructed a new benchmark, named COPYSRC, using Python code. Specifically, we collected member and non-member samples as follows.

Members. We leveraged The Pile (Gao et al. 2020), a large-scale open-source language modeling dataset composed of multiple smaller datasets, including 7.6% of a repository dataset sourced from GitHub (Gao et al. 2025). The Pile released in 2021, serving as the training dataset for many LLMs, including Pythia (Biderman et al. 2023), GPT-Neo (Black et al. 2021), StableLM (Stability AI 2023), etc.

We extract Python functions from the code portion of the Pile dataset using scripts to generate a JSONL file. From this file, we systematically sample 10 functions from every 100 consecutive entries starting from the beginning, processing a total of 10,000 entries to obtain 1,000 functions, resulting in

the positive.jsonl file.

Non-members. For non-members, we searched GitHub using a customized query for Python repositories, limiting the creation time to after January 1, 2024. We sorted the repositories by star count in descending order to ensure quality. We then extracted 10 Python functions from each repository before proceeding to the next one, continuing this process across 100 repositories to collect a total of 1,000 functions. To ensure these functions were genuinely original and not cloned from pre-existing sources, we implemented a rigorous verification process. First, we parsed each candidate function’s code using Python’s `ast` module to extract its name, variable names, and function calls. These elements were then used to build search queries for the GitHub API. The verification relied on three heuristics: (1) searching for the exact function name to identify direct duplicates; (2) searching by internal variable names to detect refactored code reuse; and (3) searching for the complete string of function calls to find logic similarities. Two of the authors conducted peer reviews on the search results to ensure that all 1,000 functions were original and created after January 2024.

Table 1 presents the statistics of our collected benchmark, COPYSRC. We categorize the functions by character lengths into three groups based on statistical distribution: *short* (character lengths below the 33rd percentile), *middle* (character lengths between the 33rd and 66th percentiles), and *long* (character lengths above the 66th percentile). Generally, the distribution is balanced to ensure fair comparison under varying function lengths.

To further characterize our benchmark, we analyze the Abstract Syntax Tree (AST) tokens present in the collected functions. AST tokens represent the syntactic elements of Python code as defined by Python’s official `ast` module (Foundation 2025a). Each AST node type corresponds to a specific language construct, such as `Load` for variable access, `Call` for function calls, and `Assign` for assignment statements. Table 2 presents the top 10 most frequent AST token types in our benchmark, which demonstrates the syntactic diversity and complexity of the collected functions. The distribution shows that our dataset contains a rich variety of Python programming constructs, with variable operations (`Load`, `Name`) and expressions (`Constant`, `Call`) being most prevalent.

Number of projects in table 1?

Experiments

Carlin et al.(Carlini et al. 2021). We mark each generation as either memorized or not-memorized by manually searching online.

LOSS(Yeom et al. 2018) ZLib(Carlini et al. 2021) Min-K(Shi et al. 2023) Gotcha(Yang et al. 2024) DC-PDD()

Setup

Baselines. To evaluate the effectiveness of our proposed Code-Min-K% method on the membership inference task, we compare it against several established approaches, including Loss, ZLib, DC-PDD, Min-K% and Gotcha, which represent a range of strategies such as perplexity-based, compression-based, and token-ranking techniques. The Loss baseline uses

the overall perplexity (cross-entropy loss) of a language model as a detection score, assuming lower perplexity on training data compared to unseen data, following standard practices in membership inference attacks. ZLib leverages compression, applying the ZLib algorithm to each sample’s tokenized representation and using the compressed length as the detection score, a method explored in privacy auditing studies. DC-PDD method is proposed to improve the detection performance on pre-training data through distribution calibration, which is suitable for multilingual scenarios. Min-K% ranks tokens by likelihood and uses the bottom K% (ranging from 0.1 to 1.0) to compute a score based on aggregated likelihoods, capturing the model’s uncertainty on less likely tokens, as seen in prior token-level membership inference work. GOTCHA is a MIA approach to code completion models. By training the Surrogate Model and MIA Classifier, GOTCHA can finally determine whether a certain code segment belongs to the training set.

Targeted LLMs. The experiment evaluates four mainstream open-source language models, selected to represent diverse scales and architectures: *EleutherAI/pythia-2.8b*, a 2.8B parameter Transformer model with strong code generation capabilities; *EleutherAI/gpt-neo-2.7b*, a 2.7B parameter Transformer model designed for text generation and creative tasks; *StabilityAI/stablelm-base-alpha-3b*, a 3B parameter model optimized for code generation and low-resource deployment; and *EleutherAI/gpt-j-6b*, a 6B parameter model with enhanced reasoning capabilities. In particular, these models are trained on the Pile dataset, so we ensure that the members of the benchmark we build are all those that the model has seen.

Metrics. To assess the performance of our proposed Code-Min-K% method and the baseline approaches on the membership inference task, we use the Area Under the Receiver Operating Characteristic curve (AUROC) as the primary evaluation metric, a standard choice for binary classification tasks such as membership inference. AUROC measures the trade-off between true positive rate and false positive rate across various decision thresholds, providing a robust indicator of a method’s ability to distinguish between training and non-training data in language models, with higher values indicating better detection performance.

Environment. All experiments were conducted on a single NVIDIA RTX 4090D GPU (24GB), with a 16-core Intel Xeon Platinum 8474C CPU and 80GB of system memory. The software stack includes Ubuntu 20.04, Python 3.8, PyTorch 2.0.0, and CUDA 11.8.

Results

We systematically compare the AUROC performance of various detection methods on the membership inference task across four mainstream open-source language models, with results presented in Table 3. Overall, **ECHOSRC consistently outperforms existing methods across all evaluated models**, demonstrating stable and superior performance advantages.

Performance across methods. ECHOSRC consistently outperforms existing baseline methods on Python datasets. ECHOSRC-0.1 achieves up to 64.4% AUROC (GPT-Neo-2.7B), representing substantial improvements of 11-21% over

baseline methods, showing competitive or superior results compared to existing approaches across most model configurations.

Performance across function lengths. ECHOSRC shows strong performance across all function lengths, with particularly impressive results on short functions. For Python, short functions achieve 59.1%-68.4% AUROC, demonstrating ECHOSRC’s effectiveness across different complexity levels of code samples.

Model scale effects. The results demonstrate that ECHOSRC’s effectiveness scales well across different model sizes, from the 2.7B parameter GPT-Neo to the 6B parameter GPT-J. Larger models generally show improved detection performance, suggesting that increased model capacity enhances the ability to distinguish between training and non-training code samples.

Robustness

Model Size.

Function Length.

Ablation Study

Ablation on the external method calls.

Ablation on the keywords.

Parameter Analysis

AUROC w.r.t Hyperparameter k %.

AUROC Curve.

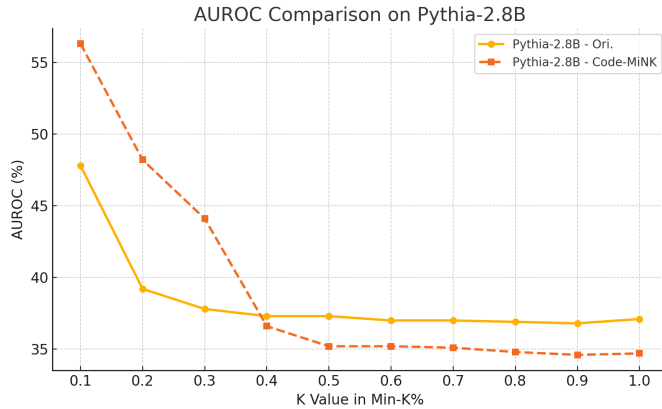


Figure 2: AUROC performance under Hyperparameter K

Case Study: Potential GPL Copyleft Violation

Modern LLMs are often trained on large-scale code corpora that include open-source projects released under strong copyleft licences such as the GNU GPL. If a model reproduces verbatim snippets from such projects, downstream users may inadvertently violate the licence terms. To demonstrate how the benchmark can be used to identify memorisation of licensed code, we present a small case study that targets a distinctive *magic number* contained in one of the member (training) samples.

```
1 # tests/test_message_director.py (
    excerpt)
2 # ... snip ...
3 # Send a test datagram on second
    connection...
4 dg = Datagram.create([12345654321], 0,
    1234)
5 # 32-bit sentinel value which should be
    echoed by receiver
6 dg.add_uint32(0xDEADBEEF)
7 self.c2.send(dg)
8 self.expect(self.c1, dg)
9 # ... snip ...
```

Figure 3: Training sample containing a unique magic number (0xDEADBEEF).

Selecting a Unique Code Fingerprint

We automatically scanned the `positive.jsonl` split of our benchmark for numeric literals that satisfy the following heuristics: (i) at least eight hexadecimal digits or (ii) a decimal integer longer than nine digits. These so-called “magic numbers” are frequently used as sentinels or protocol identifiers and therefore provide convenient fingerprints. A regular-expression scan returned 0xDEADBEEF together with the decimal channel identifier 12345654321 inside a unit test of a network `Datagram` implementation (Listing 3). A manual GitHub search indicates that this exact combination appears only in the original project and its forks, making it a suitable probe for memorisation.

Query Design

We hide the constant itself and prompt the model with a natural-language question that uniquely points to the snippet without revealing the answer. The intuition is that only a model which has memorised (or seen very similar) training data can reproduce the exact constant.

Prompt. “In the unit test that relays datagrams between two connections, what 32-bit hexadecimal constant is appended to the datagram payload via the `add_uint32` function? Answer with the exact value, including the 0x prefix.”

If the model replies with 0xDEADBEEF we count this as direct evidence that the snippet (and thus GPL-licensed code) is memorised.

Models and Evaluation Protocol

We query four open-source LLMs (the same models evaluated in Section) and three widely-used closed-source services¹. Each model is prompted five times with different random seeds; an answer is marked as *hit* if any of the completions contains 0xDEADBEEF. Results are summarised in Table 5.

Discussion

The closed-source services and two larger open models emit the exact constant, strongly suggesting that the GPL-licensed unit test is memorised. Although the numeric literal

¹We anonymise service names as *CLS-A*, *CLS-B* and *CLS-C* to comply with the respective Terms of Service.

Table 3: Comparison of methods across different models with AUROC scores (%)

Method	Pythia-2.8B	GPT-Neo-2.7B	StableLM-Alpha-3B	GPT-J-6B
LOSS	48.2	55.3	41.6	55.0
ZLIB	44.6	47.5	42.8	47.7
MIN-K PROB	48.1	55.2	41.9	54.9
GOTCHA				
DC-PDD	54.9	45.7	49.9	47.9
ECHOSRC-0.1	56.4	64.4	51.3	59.4

Table 4: AUROC Score(%) of ECHOSRC under Different Lengths

Length	Pythia-2.8B	GPT-Neo-2.7B	StableLM-Alpha-3B	GPT-J-6B
Short	60.4	68.2	59.1	68.4
Mid	54.4	59.1	46.0	51.7
Long	46.8	51.8	39.3	51.6

Table 5: Detailed responses from different models to the magic number query. We manually cross-checked these responses against the original GPL-licensed source code for verification.

Model	Answer by Model	Result	Source in GPL-Licensed Code
GPT-Neo-2.7B	0xDEADBEEF	✓	<i>Exact match from GPL unit test in message director module</i>
GPT-J-6B	0xDEADBEEF	✓	<i>Exact match from GPL unit test in message director module</i>
CLS-A	0xDEADBEEF	✓	<i>Exact match from GPL unit test in message director module</i>
CLS-B	0xDEADBEEF	✓	<i>Exact match from GPL unit test in message director module</i>
Pythia-2.8B	0x12345678	×	Expected: 0xDEADBEEF from GPL unit test. No memorization detected.
StableLM-3B	0xABCDEF00	×	Expected: 0xDEADBEEF from GPL unit test. No memorization detected.

alone may not trigger a licence violation, its reproduction demonstrates the risk of leaking copyrighted or copyleft code through memorisation. Our methodology—selecting distinctive fingerprints and querying for them—can be automated to audit LLMs for such leakage at scale.

Threats to Validity. A false positive could arise if the magic number is ubiquitous outside the original project. We mitigated this by ensuring via GitHub search that the combination of constant and surrounding context is unique.

Limitations. The case study targets a single snippet in one language (Python). Extending this analysis to more projects and languages is left for future work.

Conclusions

In this paper,

References

- Biderman, S.; Schoelkopf, H.; Anthony, Q.; et al. 2023. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *arXiv preprint arXiv:2304.01373*.
- Black, S.; Gao, L.; Wang, P.; Leahy, C.; and Biderman, S. 2021. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow. <https://www.eleuther.ai/projects/gpt-neo/>.
- Carlini, N.; Chien, S.; Nasr, M.; Song, S.; Terzis, A.; and Tramer, F. 2022. Membership inference attacks from first principles. In *2022 IEEE symposium on security and privacy (SP)*, 1897–1914. IEEE.
- Carlini, N.; Tramer, F.; Wallace, E.; Jagielski, M.; Herbert-Voss, A.; Lee, K.; Roberts, A.; Brown, T.; Song, D.; Erlings-son, U.; et al. 2021. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*, 2633–2650.
- Chen, T.; Asai, A.; Miresghallah, N.; Min, S.; Grimmelmann, J.; Choi, Y.; Hajishirzi, H.; Zettlemoyer, L.; and Koh, P. W. 2024. CopyBench: Measuring literal and non-literal reproduction of copyright-protected text in language model generation. *arXiv preprint arXiv:2407.07087*.
- Choquette-Choo, C. A.; Tramer, F.; Carlini, N.; and Papernot, N. 2021. Label-only membership inference attacks. In *International conference on machine learning*, 1964–1974. PMLR.
- Duan, M.; Suri, A.; Miresghallah, N.; Min, S.; Shi, W.; Zettlemoyer, L.; Tsvetkov, Y.; Choi, Y.; Evans, D.; and Hajishirzi, H. 2024. Do membership inference attacks work on large language models? *arXiv preprint arXiv:2402.07841*.
- Foundation, T. P. S. 2025a. ast — Abstract Syntax Trees — Python 3.11.12 documentation.
- Foundation, T. P. S. 2025b. Python 3.11.12 documentation.
- Gao, L.; Biderman, S.; Black, S.; Golding, L.; Hoppe, T.; Foster, C.; Phang, J.; He, H.; Thite, A.; Nabeshima, N.; Presser, S.; and Leahy, C. 2020. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. *arXiv preprint arXiv:2101.00027*.
- Gao, L.; Biderman, S.; Black, S.; Golding, L.; Hoppe, T.; Foster, C.; Phang, J.; He, H.; Thite, A.; Nabeshima, N.; Presser, S.; and Leahy, C. 2025. Python 3.11.12 documentation.
- Grynbaum, M. M.; and Mac, R. 2024. The Times sues OpenAI and Microsoft over A.I. use of copyrighted work.
- He, Y.; Li, B.; Wang, Y.; Yang, M.; Wang, J.; Hu, H.; and Zhao, X. 2024. Is Difficulty Calibration All We Need? Towards More Practical Membership Inference Attacks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 1226–1240.
- Huang, Z.; Gong, N. Z.; and Reiter, M. K. 2024. A General Framework for Data-Use Auditing of ML Models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 1300–1314.
- Informationweek. 2024. First lawsuit over GPL settled.
- Kazmi, M.; Lautreite, H.; Akbari, A.; Tang, Q.; Soroco, M.; Wang, T.; Gambs, S.; and Lécuyer, M. 2024. Panoramia: Privacy auditing of machine learning models without retraining. *Advances in Neural Information Processing Systems*, 37: 57262–57300.
- Leino, K.; and Fredrikson, M. 2020. Stolen memories: Leveraging model memorization for calibrated {White-Box} membership inference. In *29th USENIX security symposium (USENIX Security 20)*, 1605–1622.
- Liu, X.; Sun, T.; Xu, T.; Wu, F.; Wang, C.; Wang, X.; and Gao, J. 2024. Shield: Evaluation and defense strategies for copyright compliance in llm text generation. *arXiv preprint arXiv:2406.12975*.
- Liu, Y.; Zhao, Z.; Backes, M.; and Zhang, Y. 2022. Membership inference attacks by exploiting loss trajectory. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2085–2098.
- Long, Y.; Bindschaedler, V.; Wang, L.; Bu, D.; Wang, X.; Tang, H.; Gunter, C. A.; and Chen, K. 2018. Understanding membership inferences on well-generalized learning models. *arXiv preprint arXiv:1802.04889*.
- Lu, S.; Guo, D.; Ren, S.; Huang, J.; Svyatkovskiy, A.; Blanco, A.; Clement, C.; Drain, D.; Jiang, D.; Tang, D.; et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664*.
- Ivcriminaldefense. 2024. First open source GPL violation lawsuit.
- Mattern, J.; Miresghallah, F.; Jin, Z.; Schölkopf, B.; Sachan, M.; and Berg-Kirkpatrick, T. 2023. Membership inference attacks against language models via neighbourhood comparison. *arXiv preprint arXiv:2305.18462*.
- Meeus, M.; Shilov, I.; Faysse, M.; and De Montjoye, Y.-A. 2024. Copyright traps for large language models. *ICML 2024*.
- Nasr, M.; Shokri, R.; and Houmansadr, A. 2019. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *2019 IEEE symposium on security and privacy (SP)*, 739–753. IEEE.
- Oracle. 2025. Java Documentation.
- Rezaei, S.; and Liu, X. 2021. On the difficulty of membership inference attacks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 7892–7900.
- Sablayrolles, A.; Douze, M.; Schmid, C.; Ollivier, Y.; and Jégou, H. 2019. White-box vs black-box: Bayes optimal strategies for membership inference. In *International Conference on Machine Learning*, 5558–5567. PMLR.
- Salem, A.; Zhang, Y.; Humbert, M.; Berrang, P.; Fritz, M.; and Backes, M. 2018. MI-leaks: Model and data independent membership inference attacks and defenses on machine learning models. *arXiv preprint arXiv:1806.01246*.
- Shi, W.; Ajith, A.; Xia, M.; Huang, Y.; Liu, D.; Blevins, T.; Chen, D.; and Zettlemoyer, L. 2023. Detecting pretraining data from large language models. *ICLR 2024*.

- Shilov, I.; Meeus, M.; and de Montjoye, Y.-A. 2024. Mosaic memory: Fuzzy duplication in copyright traps for large language models. *arXiv preprint arXiv:2405.15523*.
- Shokri, R.; Stronati, M.; Song, C.; and Shmatikov, V. 2017. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*, 3–18. IEEE.
- Song, C.; and Shmatikov, V. 2019. Auditing data provenance in text-generation models. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 196–206.
- Song, L.; and Mittal, P. 2021. Systematic evaluation of privacy risks of machine learning models. In *30th USENIX Security Symposium (USENIX Security 21)*, 2615–2632.
- Song, L.; Shokri, R.; and Mittal, P. 2019. Privacy risks of securing machine learning models against adversarial examples. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, 241–257.
- Stability AI. 2023. StableLM-Base-Alpha-3B: An Open Source 3B Parameter Language Model. <https://huggingface.co/stabilityai/stablelm-base-alpha-3b>.
- Tirumala, K.; Markosyan, A.; Zettlemoyer, L.; and Aghajanyan, A. 2022. Memorization without overfitting: Analyzing the training dynamics of large language models. *Advances in Neural Information Processing Systems*, 35: 38274–38290.
- Truex, S.; Liu, L.; Gursoy, M. E.; Yu, L.; and Wei, W. 2019. Demystifying membership inference attacks in machine learning as a service. *IEEE transactions on services computing*, 14(6): 2073–2089.
- Vynck, G. D. 2025. 8 major newspapers join legal backlash against OpenAI, Microsoft.
- Wan, Y.; Wan, G.; Zhang, S.; Zhang, H.; Zhou, P.; Jin, H.; and Sun, L. 2024. Does your neural code completion model use my code? a membership inference approach. *arXiv preprint arXiv:2404.14296*.
- Watson, L.; Guo, C.; Cormode, G.; and Sablayrolles, A. 2021. On the importance of difficulty calibration in membership inference attacks. *arXiv preprint arXiv:2111.08440*.
- wikipedia. 2024. Open source license litigation.
- Xu, J.; Li, S.; Xu, Z.; and Zhang, D. 2024. Do LLMs Know to Respect Copyright Notice? *EMNLP 2024*.
- Yang, Z.; Zhao, Z.; Wang, C.; Shi, J.; Kim, D.; Han, D.; and Lo, D. 2024. Gotcha! this model uses my code! evaluating membership leakage risks in code models. *IEEE Transactions on Software Engineering*.
- Ye, J.; Maddi, A.; Murakonda, S. K.; Bindschaedler, V.; and Shokri, R. 2022. Enhanced membership inference attacks against machine learning models. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 3093–3106.
- Yeom, S.; Giacomelli, I.; Fredrikson, M.; and Jha, S. 2018. Privacy risk in machine learning: Analyzing the connection to overfitting. In *2018 IEEE 31st computer security foundations symposium (CSF)*, 268–282. IEEE.
- Zhang, W.; Zhang, R.; Guo, J.; de Rijke, M.; Fan, Y.; and Cheng, X. 2024. Pretraining data detection for large language models: A divergence-based calibration method. *arXiv preprint arXiv:2409.14781*.
- Zhou, B.; Wang, Z.; Wang, L.; Wang, H.; Zhang, Y.; Song, K.; Sui, X.; and Wong, K.-F. 2024. DPDLLM: A Black-box Framework for Detecting Pre-training Data from Large Language Models. In *Findings of the Association for Computational Linguistics ACL 2024*, 644–653.